ECEN 5113

HW: 2

Part B

Saira Khan

September 24, 2025

Procedure:

Vivado was used to create, synthesize, simulate, and test each design made in this section. An RTL project is created in Vivado, and design sources and simulation files are added to each project. A behavioral simulation is run to generate a waveform (timing schematic). The default FPGA part used is xc7k70tfbv67-1.

For this lab, an ALU module with 8 functions was created. A 3-bit control input (C2, C1, and C0 in Figure 2) chooses between one of the 8 functions for the ALU to execute based on Table 1. A and B are four-bit inputs, carry-in is a one-bit input, output is a four-bit output, and carry-out is a one-bit output. The structural Verilog model was used to create the first function, add, by using logic gates and wires to create a full adder circuit. For example, the first gate declared is an XOR gate that has an output of w1 (wire 1), and inputs of a and b. A wire for the output of the adder and the carry out of the adder is created to store those values. Four full adders are then instantiated in the main ALU module to create the four-bit adder. The main ALU module contains a dataflow Verilog model of And, which is shown by the assign statement and the boolean operator (&). 4-bit wires are created for each output of each of the eight functions. The behavioral model of Verilog is used to describe the rotate left function using the procedural statement always. To rotate the value A, the output rol_out is assigned to A[2:0], A[3], which puts the first (leftmost) bit to the back (rightmost). Logical and arithmetic operators are used to create the rest of the functions. The subtraction function is a little different from the other functions because the “borrow” is taken into account. A 5-bit output wire is created in order to store the borrow bit, and A-B-cin is the output. The actual result is stored in a 4-bit wire, and the borrow bit is stored separately (it is defined as the carry out of the ALU). The main ALU logic is stored in an always procedural statement. In order to implement the control logic, case statements are used. Each case statement corresponds to one of the 8 functions (with a default case that sets the outputs to 0). The outputs and carry out values correspond to the wires declared earlier, and some carry out values are declared as 0 because the function does not output a carry out. A testbench is then created to test the logic of this ALU. The inputs and outputs are declared, and the ALU module is instantiated as “d”. An initial block is used to simulate the logic. Different inputs of A and B are tested along with different inputs of the control, such that each function is tested at least once. The output and inputs are displayed using the $display function. A delay of 10 time units is placed between the change of inputs and the display of the variables to give the program time to update.

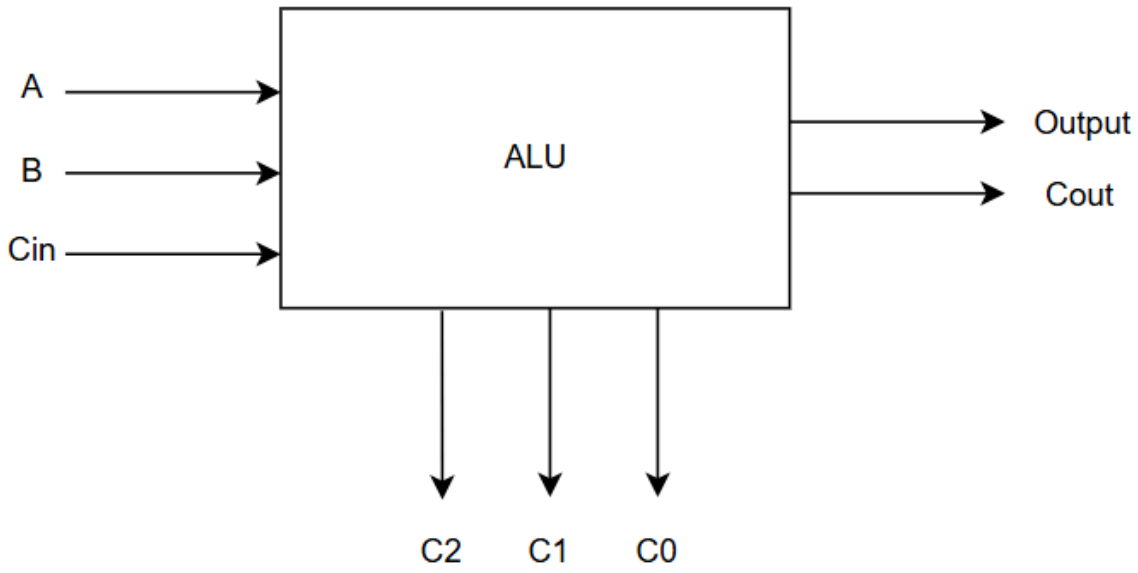

Figure 1: Block Diagram of ALU

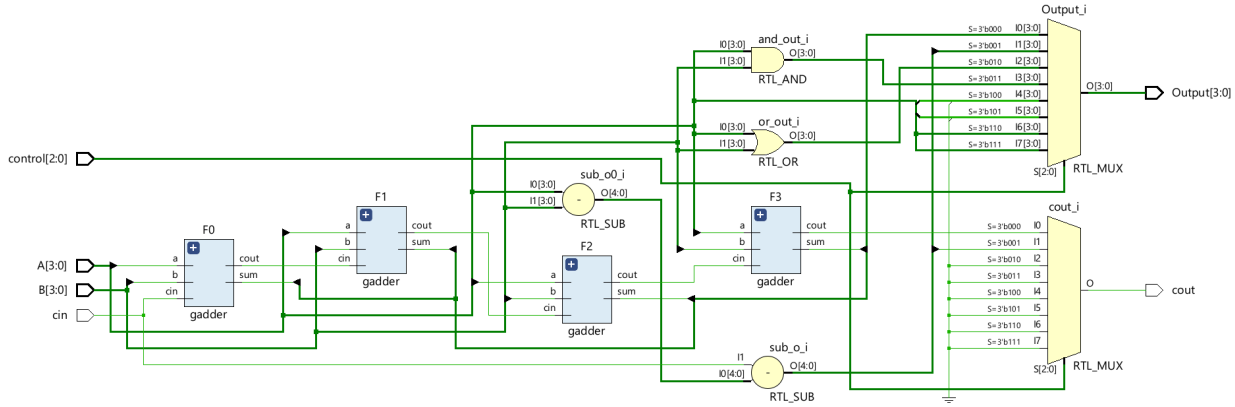

Figure 2: Schematic of ALU

Table 1: ALU Control and Corresponding Instructions

Control	Instruction
000	Add
001	Subtract
010	Or
011	And
100	Shl
101	Shr
110	Rol
111	Ror

Observations:

Before creating an individual output for each function, I had assigned the outputs to the variable Output. Once I tried actually simulating the module, I realized that I could not do that. Each operation needed its own wire. The same signal can not be driven from multiple sources, so each function needed its own output, which was then assigned to the actual Output variable in the always procedural statement.

Results:

Figures 3 and 4 show the output of the ALU synthesized and simulated using Vivado. Figure 3 shows how the ALU module testbench was implemented and confirms that the logic of the ALU works. Figure 4 shows the waveform of the ALU, with the x-axis as time and the y-axis showing the variables. The testbench runs through each of the 8 functions and tests the borrow logic implemented by the subtractor. The second case shows the output being a 2's complement because A-B gives a negative number, and the carry out is equal to 1 because a borrow was involved.

```
# run 1000ns
time=10000 | A=0000 | B=0000 | cin=0 | control=000 | Output=0000 | cout=0
time=20000 | A=0001 | B=1001 | cin=0 | control=001 | Output=1000 | cout=1
time=30000 | A=0001 | B=0001 | cin=0 | control=001 | Output=0000 | cout=0
time=40000 | A=0101 | B=1011 | cin=0 | control=010 | Output=1111 | cout=0
time=50000 | A=1111 | B=0011 | cin=0 | control=011 | Output=0011 | cout=0
time=60000 | A=1001 | B=0101 | cin=0 | control=100 | Output=0010 | cout=0
time=70000 | A=1001 | B=0101 | cin=0 | control=100 | Output=0010 | cout=0
time=80000 | A=1101 | B=1001 | cin=0 | control=101 | Output=0110 | cout=0
time=90000 | A=0101 | B=1110 | cin=0 | control=110 | Output=1010 | cout=0
time=100000 | A=1000 | B=1100 | cin=0 | control=111 | Output=0100 | cout=0
```

Figure 3: Output of ALU from Vivado Simulation

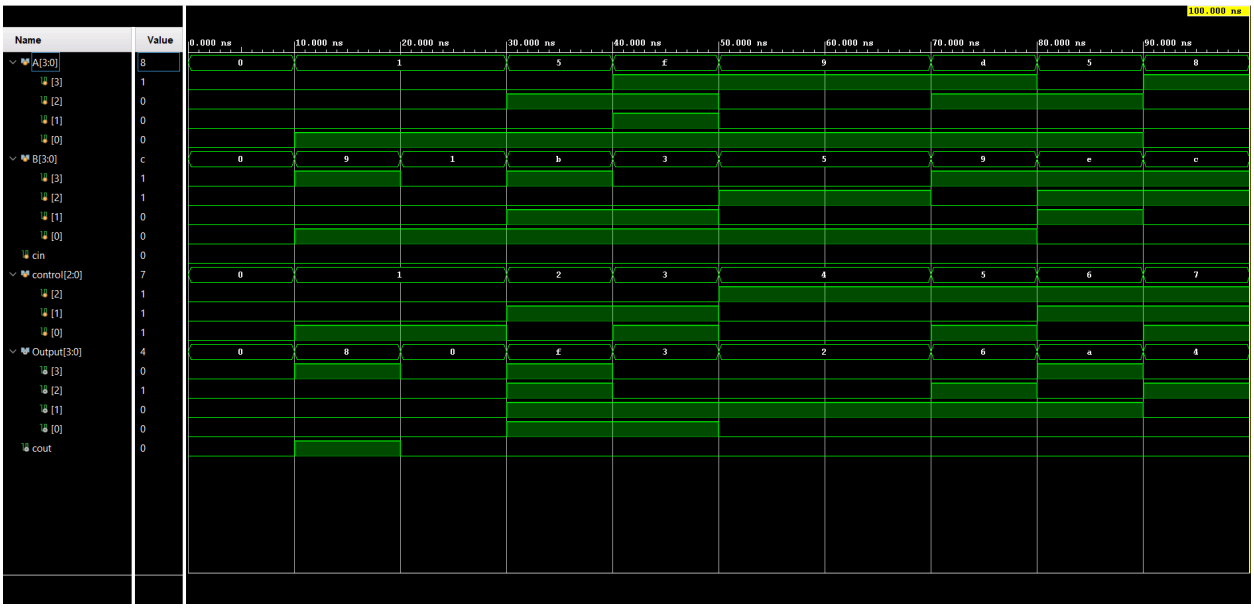

Figure 4: Waveform of ALU from Vivado Simulation

Conclusion:

For this lab, a 4-bit input ALU Verilog module was created with eight functions selected by the 3-bit control input. All three models of Verilog were used in this module. Structural Verilog was used to create the adder function, dataflow model was used to make the and function, and behavioral Verilog was used to create the rotate left function. The different models of Verilog can be mixed together because they all describe the same underlying digital hardware. Verilog was designed as a hierarchical modeling language where top-levels may use behavioral, middle levels may use dataflow, and bottom-levels may use structural.

Appendix:

ALU Verilog Code:

```
//gate level adder - 1 bit

//define module

module gadder(

   input a,

   input b,

   input cin,

   output sum,

   output cout

);


   //define internal wires

   wire w1, w2, w3;


   //sum of a and b

   xor g1(w1, a, b);


   //and of a and b

   and g2(w2, a, b);


   //sum w1 (a + b) and cin

   xor g3(sum, w1, cin);
```

```
    //and w1 (sum a + b) + cin
    and g4(w3, w1, cin);

    //final carry
    or g5(cout, w2, w3);

endmodule
```

```
//4 bit ALU
//define module
module alu(
    //define inputs and outputs
    input [3:0] A,
    input [3:0] B,
    input cin,
    input [2:0] control,
    output reg [3:0] Output,
    output reg cout
);

    //adder

    //declare wires
    wire c1, c2, c3;
```

```
//wires for adder output
wire [3:0] add_out;
wire add_cout;

//instantiate 4 full adders
gadder F0 (A[0], B[0], cin, add_out[0], c1);
gadder F1 (A[1], B[1], c1, add_out[1], c2);
gadder F2 (A[2], B[2], c2, add_out[2], c3);
gadder F3 (A[3], B[3], c3, add_out[3], add_cout);


//and - dataflow
wire [3:0] and_out;//output of and
assign and_out = A&B; //and operation

//or
wire [3:0] or_out;
assign or_out = A | B;


//behavioral rol
reg [3:0] rol_out;
always @*
begin
//rotate left by 1
rol_out = {A[2:0], A[3]};
end
```

```
//ror
wire [3:0] ror_out = {A[0], A[3:1]};

//shift left
wire [3:0] shl_out = {A[2:0], 1'b0};

//shift right
wire [3:0] shr_out = {1'b0, A[3:1]};

//subtraction
wire [4:0] sub_o = A - B-cin; // 5 bits to capture borrow bit
wire [3:0] sub_out = sub_o[3:0]; //result
wire borrow = sub_o[4]; //borrow bit (msb)





//combinational logic selection

always @* begin
   //set outputs
   Output = 4'b0000;
   cout = 1'b0;



   case(control)
   //add
   3'b000: begin
      Output = add_out;
      cout = add_cout;
```

```
end

//sub
3'b001: begin
    Output = sub_out;
    cout= borrow;

end

//or
3'b010:begin
    Output = or_out;
    cout = 1'b0;
end

//and
3'b011:begin
    Output = and_out;
    cout = 1'b0;
end

//shl
3'b100: begin
    Output = shl_out;
    cout = 1'b0;

end

//shr
3'b101:begin
```

```
        Output= shr_out;
        cout = 1'b0;

      end

      //rol
      3'b110:begin
        Output = rol_out;
        cout = 1'b0;
      end



      //ror
      3'b111:begin
        Output = ror_out;
        cout = 1'b0;
      end

      default:begin
        Output = 4'b0000;
        cout = 1'b0;
      end
      endcase
    end
endmodule
```

ALU Testbench Code:

```
//testbench
//define module
```

```
module alu_tb;
    //define inputs/outputs
    reg [3:0] A;
    reg [3:0] B;
    reg cin;
    reg [2:0] control;
    wire [3:0] Output;
    wire cout;

    //instantiate the module
    alu d(A, B, cin, control, Output, cout);




    //initial block for simulating

    initial
        begin
        //set inputs
        A = 4'b0000;
        B = 4'b0000;
        cin = 1'b0;
        control= 3'b000;




        #10; //delay
        $display("time=%0t | A=%4b | B=%4b | cin=%b | control=%3b | Output=%4b | cout=%b",
         $time, A, B, cin, control, Output, cout);
```

```
//test different inputs to check alu working

//sub w borrow
A = 4'b0001; B=4'b1001; control=3'b001; cin=1'b0;
#10;
$display("time=%0t | A=%4b | B=%4b | cin=%b | control=%3b | Output=%4b | cout=%b",
 $time, A, B, cin, control, Output, cout);

//sub w/o borrow
A = 4'b0001; B=4'b0001; control=3'b001; cin=1'b0;
#10;
$display("time=%0t | A=%4b | B=%4b | cin=%b | control=%3b | Output=%4b | cout=%b",
 $time, A, B, cin, control, Output, cout);


A = 4'b0101; B=4'b1011; control=3'b010; cin=1'b0;
#10;
$display("time=%0t | A=%4b | B=%4b | cin=%b | control=%3b | Output=%4b | cout=%b",
 $time, A, B, cin, control, Output, cout);


A = 4'b1111; B=4'b0011; control=3'b011; cin=1'b0;
#10;
$display("time=%0t | A=%4b | B=%4b | cin=%b | control=%3b | Output=%4b | cout=%b",
 $time, A, B, cin, control, Output, cout);

A = 4'b1001; B=4'b0101; control=3'b100; cin=1'b0;
#10;
$display("time=%0t | A=%4b | B=%4b | cin=%b | control=%3b | Output=%4b | cout=%b",
 $time, A, B, cin, control, Output, cout);;
```

```
    A = 4'b1001; B=4'b0101; control=3'b100; cin=1'b0;
    #10;
    $display("time=%0t | A=%4b | B=%4b | cin=%b | control=%3b | Output=%4b | cout=%b",
     $time, A, B, cin, control, Output, cout);


    A = 4'b1101; B=4'b1001; control=3'b101; cin=1'b0;
    #10;
    $display("time=%0t | A=%4b | B=%4b | cin=%b | control=%3b | Output=%4b | cout=%b",
     $time, A, B, cin, control, Output, cout);


    A = 4'b0101; B=4'b1110; control=3'b110; cin=1'b0;
    #10;
    $display("time=%0t | A=%4b | B=%4b | cin=%b | control=%3b | Output=%4b | cout=%b",
     $time, A, B, cin, control, Output, cout);


    A = 4'b1000; B=4'b1100; control=3'b111; cin=1'b0;
    #10;
    $display("time=%0t | A=%4b | B=%4b | cin=%b | control=%3b | Output=%4b | cout=%b",
     $time, A, B, cin, control, Output, cout);


    $finish; //finish simulation
  end

endmodule
```